

The 6th place solution (CV 0.39)

Rank	6
Team	Vadim Timakin
Author	Vadim Timakin
Topic ID	549582
Version	Original

Source: <https://www.kaggle.com/competitions/um-game-playing-strength-of-mcts-variants/discussion/549582>

Retrieved with Kaggle CLI on 2026-07-03.

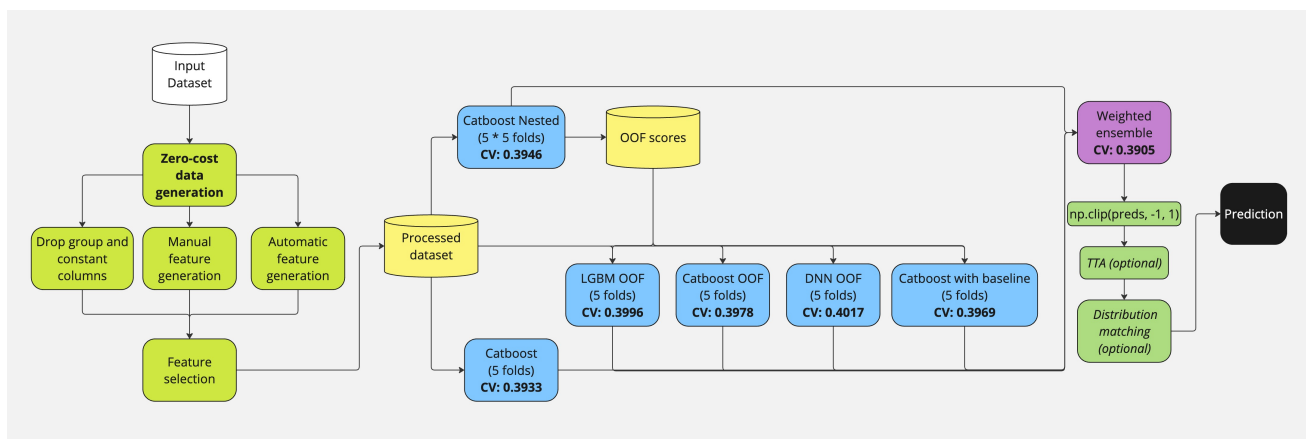
Intro

First of all, I would like to thank the organizers of this competition, this task was very interesting to solve and I learned a lot. Also, many thanks and respect to everyone who shared their ideas during this competition. And last but not least, congratulations to all the winners!

1. Summary

My solution combines the power of zero-cost data generation, good modeling, both manual and automatic feature generation, ensembling with two-staged models and also has some risky parts that I'll try to justify further.

The general pipeline is presented below.



2. Validation strategy

Starting with simple GroupKFold with GameRulesetName as a group as anyone else in this competition was a good baseline, however wasn't stable enough. First modification that I did was adding a stratification by target, after this I also added agent1 column to the stratify criteria.

Even with this CV approach I didn't get a complete correlation between CV and public LB or filled the gap between CV and LB (which in my opinion shouldn't be a concern in competitions which use a group type of stratification between train and test set), however it still was more stable and showed more expected results locally with different approaches through this competitions, it also showed consistent results with different seeds on both CV and public LB.

It's also important to keep the split always the same and validate only on source data which become the most actual with the new data generation.

Using more than 5 folds was leading to a minor score boost yet was less stable.

3. Data part

3.1. Preprocessing

1. Drop constant columns.
2. Drop GameRulesetName, LudRules and EnglishRules.
3. Memory optimization.

I also experimened with one-hot / label / target encoding for categorical features and scaling / binarizing / normalizing numerical ones to improve model perfomance and reduce overfitting, but it was ineffective in all the cases except for training a DNN model.

3.2. Zero-cost data generation

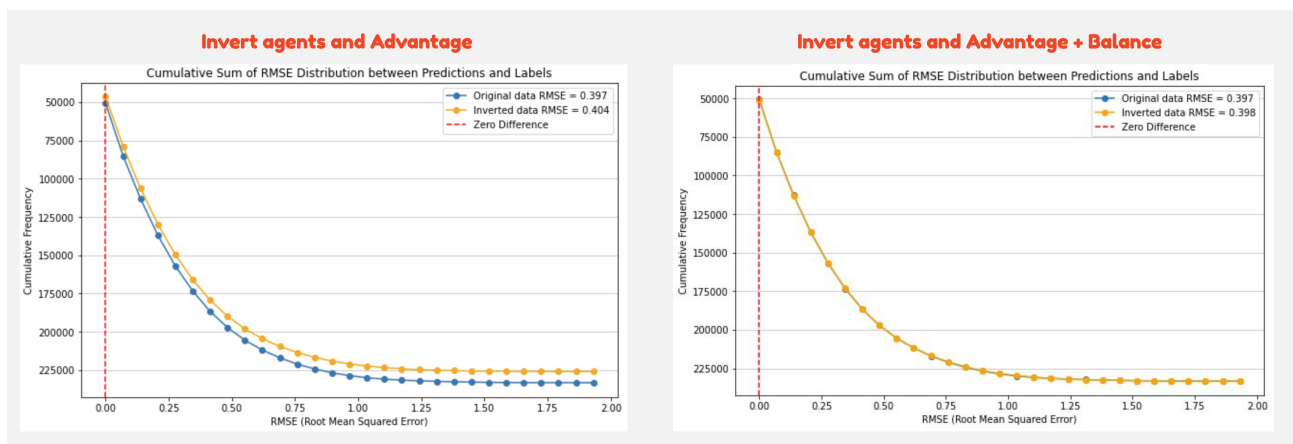
This part is based on both logic and some empirical assumptions.

Initially, the idea to generate more data comes from the fact that we have some game depending and player depending features which also depend on the order. First, I tried to swapping agent strings (here and further I always invert the target as well) and AdvantageP1 column thinking that it represents the probability of winning of a certain agent over another one, which already gave a solid boost for my CV and LB (it's important to validate only on original data). Soon, I realised that this column actually represents the probability of the first player winning the second one in this game where both players play randomly. So this feature logically shouldn't be inverted and I removed inversion for it, but it actually decreased the score! My interpretation of this is that while it's logically correct to only swap agents and target, such transformation doesn't bring any new signal to the data and therefore leads to the worse results.

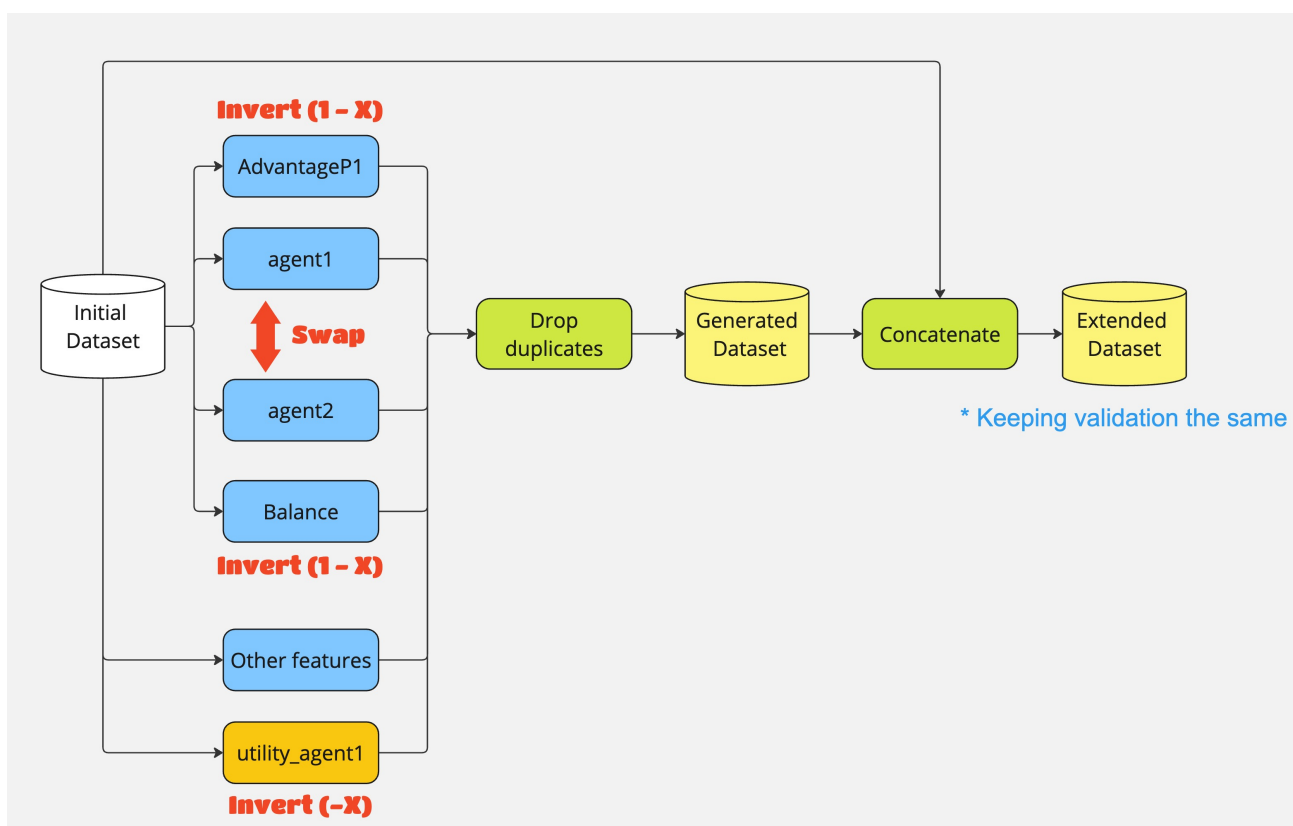
After this I still had a gap between my original and inverted OOF predictions scores, so I was looking for ways to fill it by trying to invert other features. One of this features was the Balance, which had the highest feature importance after the AdvantageP1.

	feature	importance
0	AdvantageP1	15.530264
1	Balance	4.956622
2	p2_selection	4.010179
3	p2_agent	2.901193
4	p2_playout	2.842518

So I tried to invert this feature as well and that's what filled the gap almost completely. Besides this, I also tried to invert even more features, but the rest of them had lower feature importance and didn't bring any new signal to the data. Below is the demonstration of the gap for the case without inverting the Balance column and with inverting it as well.



The overall data generation pipeline is illustrated below.



Besides this I tried other methods of new data generation, like agents playing with themselves, but it was less valuable.

When I filled the error gap between original and generated data, it also gave me an opportunity to use TTA which could boost my score on 0.001-0.002 points. However, it's not used in my final submission.

Inspired by the results described above, I also tried doing pseudo-labeling by training another model without group split criteria and then generating the new data with it, but it didn't improve the score.

3.3. Manual feature generation

I tried to generate a lot of features manually, most of them didn't work, but here a bunch of successful ones.

1) The first set of features comes from simply splitting the agent strings.

```
df = df.with_columns(
    pl.col()..extract(, ).alias(),
    pl.col()..extract(, ).alias(),
    pl.col()..extract(, ).alias(),
    pl.col()..extract(, ).alias(),
    pl.col()..extract(, ).alias(),
    pl.col()..extract(, ).alias(),
    pl.col()..extract(, ).alias(),
    pl.col()..extract(, ).alias(),
    pl.col()..extract(, ).alias(),
)
```

2) The second set of features is taken from [this notebook](#) by @litsea.

```
df = df.with_columns([
    (pl.col() / (pl.col() + )).alias(),
    (pl.col() / (pl.col() + )).alias(),
    (pl.col() / (pl.col() + )).alias(),
    (pl.col() / (pl.col() + )).alias(),
    (pl.col() / (pl.col() + )).alias(),
    (pl.col() / (pl.col() + )).alias(),
    (pl.col() / (pl.col() + )).alias(),
    (pl.col() / (pl.col() + )).alias(),
    (pl.col() * pl.col()).alias(),
    (pl.col() + pl.col()).alias(),
    (pl.col() / (pl.col() + )).alias(),
    (pl.col() / (pl.col() + )).alias(),
    (pl.col() / (pl.col() + )).alias(),
    (pl.col() + pl.col() + pl.col()).alias(),
    (pl.col() +
        pl.col() +
        pl.col() +
        pl.col()).alias(),
    (pl.col() / (pl.col() + )).alias(),
    (pl.col() / (pl.col() + )).alias(),
    (pl.col() / (pl.col() + )).alias(),
])
```

3) The third set of features I call cross-agent features, it captures the interactions between different subtypes of agents.

```
df[] = df[]
df[] = df[]

cols = [, , , , ]
c1 cols:
c2 cols:
    df[] = df[].astype() + df[].astype()
```

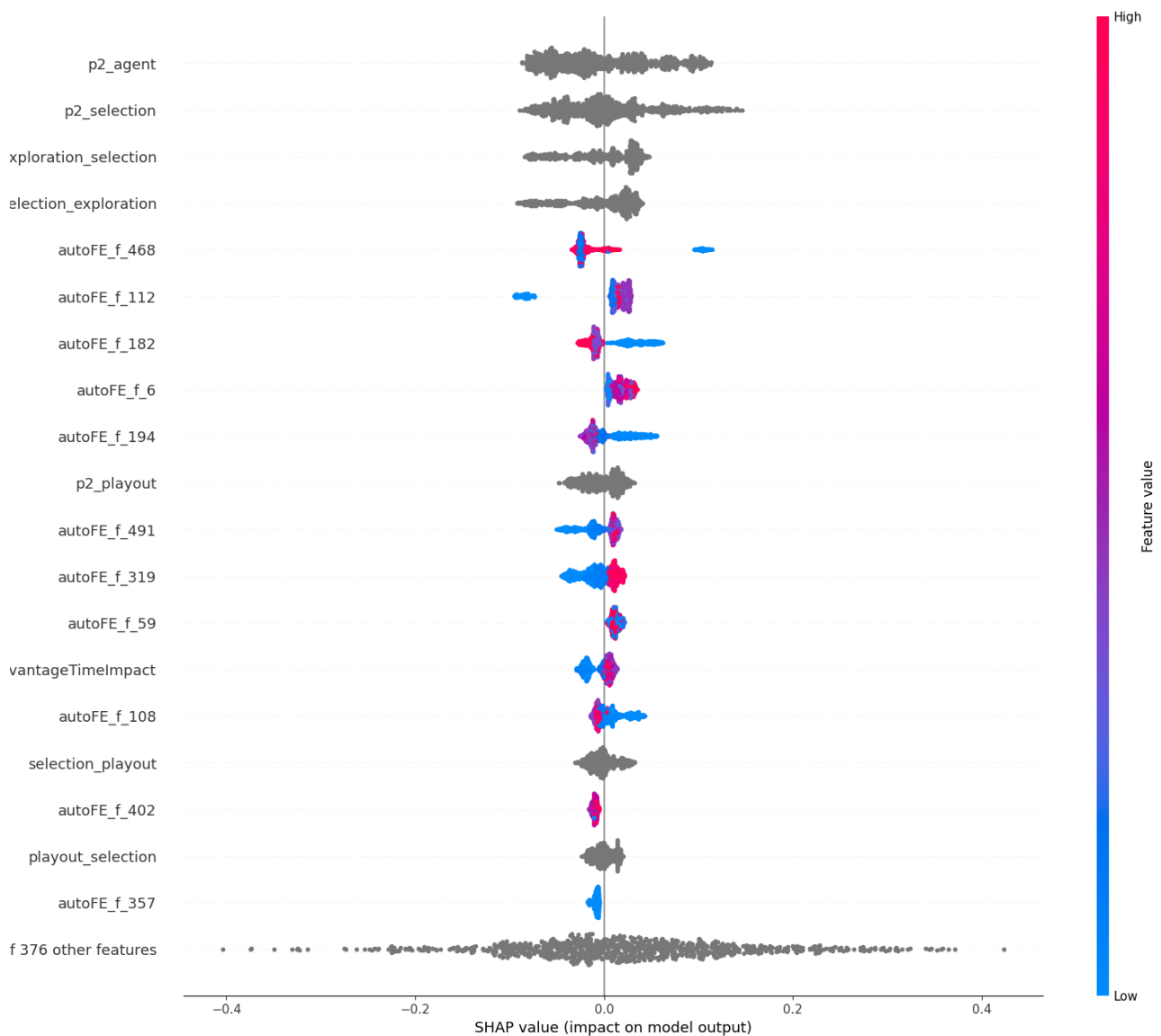
Talking about the features generated from the columns with the rules. For me, they all turned out to be ineffective. I tried using TF-IDF, counting various indexes, but it did not give an increase in score.

3.4. Automatic feature generation

I used [OpenFE](#) library for automatic feature generation. It's a great algorithm, however it didn't work out-of-box as I expected, so I had to make a list of necessary changes to make it work in this competition. Basically, I went through the next steps:

1. Fix bugs related to multiprocessing.
2. Fix bugs related to data types.
3. Adjust logging for debug purposes.
4. Replace StratifiedKfold with StratifiedGroupKfold.
5. Sample 10% of by original data (considering both stratify and group criteria) for running the algorithm.
6. Run the algorithm (around 30 hours on 10% subsample and 1 CPU).
7. Select top-500 the best features.
8. Remove features aggregated by groups and freq features as they produce unstable results in public LB.
9. Fill NaNs with "-100" value.
10. Save features and metadata for generating them on inference.

The graph below shows that many of these features turned out to be quite informative.



3.5. Feature selection

After all the feature generations I left only about 400 columns based on the CV importance. It didn't have any impact neither on CV nor on public LB and was done just for saving memory and faster training purposes.

Besides that I tried to drop features not only based on feature importance but also based on other factors: correlation, variance, number of unique values per group, etc. But it didn't work.

4. Modeling

4.1. Approach

From the very beginning to the end of the competition, Catboost showed the best results for me, while other models were worse, so I decided to use a two-stage approach to build the ensemble.

I considered this problem as a regression task and trained all my models to optimize RMSE metric. I tried to solve this task as a classification one and optimizing QWK, but got worse results. Anyways, since we have discrete labels in training data, it's possible to find a way of rounding predictions that would lead to a performance boost. I did some experiments with post-processing, where I was looking for some thresholds for rounding my predictions, however the results were unstable so I decided not to use it in my final solution.

I also tried different regularizations to achieve more balanced feature importances but it didn't lead to any improvements as well.

4.2. Catboost

The best single model. The parameters are adjusted manually.

```
catboost_params = {
    : ,
    : ,
    : ,
    : ,

    : ,
    : ,

    : ,
    : ,
    : ,

    : ,
    : ,
}
```

4.3. Two-staged models

LGBM, XGB and DNN models were consistently worse in this competition, so I decided to use catboost OOF predictions as another feature for these models.

1) First, I needed to collect those OOF predictions properly. Using simple outputs of 5 folds model split didn't work and caused leakage so I used a **Nested CV Catboost model (5*5 folds)** for this. The reason to do so is described well in [this discussion](#) and [this notebook](#) by [@martinapreusse](#).

2) Then I trained **Catboost OOF** and **LGBM OOF** models. Parameters for Catboost are the same as above, parameters for LGBM model presented below, I didn't tune them a lot and there is a huge room for improvement here.

[illegible]


```
:  
}
```

3) Next I trained another **Catboost** model using OOF predictions, but this time not as a feature but as a **baseline initialization**. An example can be found [here](#).

4) Finally, I trained a **DNN model**. I tried a lot of different architecture variations, but ended up with a classic MLP with modifications:

```
. Embedding layer () for categorical features.  
. Quantile transformer for numerical features.  
. OOF feature as it is.  
. Concat -> MLP (Dropout=, Hardswish, -> -> -> -> -> -> ).  
. MADGRAD optimizer and RMSE loss for optimization.
```

4.4. Other models

Besides that I tried to train another meta-models that would predict number of games played, corner cases and draws, however it didn't lead to any significant improvements. XGBoost models weren't effective as well.

4.5. Ensembling

After testing various ensembling approaches I just ended up choosing weighted ensemble with positive weights selected by the scipy minimize function based on CV score.

5. Matching the distribution

Since the beginning of the competition, I noticed that at least the training and public test dataset have a different distribution. This was indirectly confirmed by big gap between LB and CV scores and LB probing done by other participants, for example in [this discussion](#) by @tomooiubushi. I also saw a lot of data shifting and scaling in public notebooks. I decided to focus on my CV in this competition and come back to this closer to the end of competition.

At the end I had a stable ensemble with a good CV score, so I decided to use it as my stable submission without any post-processing, while reserving another one for a more risky approach. I took one of my notebooks which had the best public score and did some probing as well. I tried different clipping, shifting and scaling technique in order to match the public distribution the best.

I was able to improve my best public score from 0.421 to 0.417 using the following adjustment:

```
predictions = np.clip(predictions * , -, )
```

What's interesting is that applying the same adjustment for my best CV ensemble improved the public score only from 0.424 to 0.423.

Basically, the idea behind this is that the private test dataset either has completely new games or has games that intersect with games from public test dataset (or similar to them). I tried to cover both of the cases.

6. Final results

The submission aligned the best with the public LB scored the best on the private LB as well.

Model	CV	Public LB	Private LB
Catboost Classic	0.3933	0.426	0.432
Catboost Nested	0.3946	0.426	0.433
Catboost with OOF	0.3978	0.425	0.431
Catboost with OOF as baseline	0.3969	0.426	0.433
LGBM with OOF	0.3996	0.426	0.432
DNN with OOF	0.4017	0.426	0.432
Best CV ensemble	0.3905	0.424	0.430
Best public (Catboost + Distribution Match)	0.3995	0.417	0.422

7. What can also be beneficial and what is not

- **Meaningful LB probing** - could be useful here, but I started doing it only closer to the end of the competition, yet I see many people with hundreds of submissions and looking forward to find out their potential approaches.
- **LLM for rules parcing** - it seems like there just aren't enough unique rules for training a language model, so I didn't do it, yet again looking forward to find out if anyone did it successfully.
- **Pseudo-labeling on test data** - could be possible and valuable here if the test dataset wasn't given in batches.
- **Handle randomness better** - there are obviously deterministic games and probability based ones (for example with dice(s)), handling the second type of games better and matching the possible distribution the smarter way could be beneficial here.
- **Classic data generation** - takes a lot of compute, but must be valuable here.

Thanks for reading and good luck at the next competitions, see you there!

Code: <https://github.com/vadimtimakin/Kaggle-MCTS-6th-place-solution>

EDIT: typos

Supplemental Q&A

Aurora Rabbit · 2024-12-03T01:21:12.967000

Congratulations!

Your DNN model seems to be performing well. My DNN model, on the other hand, appears to rely primarily on around the top 20 features.

Thanks!

In fact it performed kinda weak without the OOF feature generated by catboost. Also using this feature at first caused overfitting and low score as well, which was solved by using high dropout rate.